

```

playerpos[1]+=5
if keys[1]:
    playerpos[0]-=5
elif keys[3]:
    playerpos[0]+=5

```

This code checks which keys are being pressed, and adds or subtracts from the player's x or y position (depending on the key pressed) to move the player. Run the game and you would get a player just like before. Press WASD. Yay! It works.

Turn bunny with mouse. Your bunny can now move (up, down, left and right) when you press appropriate keys on the keyboard. But can you use the mouse to rotate the bunny to face the direction of your choice? It is simple to implement this using trigonometry. Take a look at the illustration in Fig. 3.

In the above image, if (5,3) is the position of the bunny and (2,4) is the current position of the mouse, you can find the rotation angle (z) by applying atan2 trigonometric function to the difference in distances between the two points. Of course, once you know the rotation angle, you can simply rotate the bunny accordingly.

You use this stuff all the time in game programming. To do this, use PyGame Surface.rotate(degrees) function. Keep in mind that z value is in radians. The atan2 function comes from Python Maths library. Add this library to the import library section #1 first.

Replace the last line in draw the screen element section #6 (player, blit line) with the following code:

```

# 6.1 - Set player position and rotation
position = pygame.mouse.get_pos()
angle = math.atan2(position[1]-
    (playerpos[1]+32),position[0]-
    (playerpos[0]+26))
playerrot = pygame.transform.
    rotate(player, 360-angle*57.29)

```

In the above code, first, get mouse and player positions and feed those into atan2 function. Then, convert the angle received from atan2 function from radians to

degrees (multiply radians by approximately 57.29 or $360/2\pi$). Since the bunny will be rotated, its position will change.

Calculate the new bunny position and display the bunny on the screen.

Run the game again. If you use just WASD keys, then the game would behave exactly like before. But if you move your mouse, the bunny rotates, too.

Shoot, bunny, shoot! Now that your bunny's moving around, it is time to add a little more action. How about letting the bunny shoot enemies using arrows? This step is a bit more complicated because you have to keep track of all arrows, update, rotate and delete these when these go off screen.

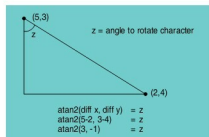


Fig. 3: Trigonometric illustration

Add the necessary variables to the end of initialisation section, section #2:

```

acc=[0,0]
arrows=[]

```

The first variable keeps track of the player's accuracy and the second tracks all arrows. The accuracy variable is essentially a list of the number of shots fired and the number of moles hit. Later, we use this information to calculate an accuracy percentage.

Load the arrow image at the end of section #3 using the following:

```

arrow = pygame.image.load ("resources/
    images/bullet.png")

```

When a user clicks the mouse, an arrow should fire. Add the following to the end of section #8 as a new event handler:

```

if event.type==pygame.MOUSEBUTTONDOWN:
    position=pygame.mouse.get_pos()
    acc[1]+=1

```

This code checks if the mouse was clicked, and if it was, it gets the mouse's position and calculates arrow rotation based on the rotated player position and the cursor position. This rotation value is stored in arrows array.

Draw arrows on screen. Add the following code right after section #6.1:

```

# 6.2 - Draw arrows
for bullet in arrows:
    index=0
    velx=math.cos(bullet[0])*10
    vely=math.sin(bullet[0])*10
    bullet[1]=velx
    bullet[2]=vely
    if bullet[1]<=64 or bullet[1]>640 or
    bullet[2]<=64 or bullet[2]>480:
        arrows.pop(index)
    index+=1
for projectile in arrows:
    arrow1 = pygame.transform.
        rotate(arrow, 360-projectile
            [0]*57.29)
    screen.blit(arrow1,(projectile[1],
        projectile[2]))

```

The velx and vely values are calculated using basic trigonometry. Ten is the speed of the arrows. The 'if' statement checks if the bullet is out of bounds and, if so, deletes the arrow. The second 'for' statement loops through the arrows and draws these with the correct rotation. Run the program; you would have a bunny that shoots arrow (Fig. 4) when you click the mouse.

Take up arms, moles! You have a castle and a hero who can move and shoot. So what is missing? Enemies who attack the castle that the hero can shoot at!

In this step, you will create randomly-generated moles that run at the castle. There will be more moles as the game progresses. So let us see what you will need to do so. Add bad guys to a list of array. Update the bad guy array frame, show the bad guys on the screen.

Add the following code to the end of section #2:

```

badtimer=100
badtimer1=0

```